

Web アプリ チーム開発

はじめての一步

GDG on Campus University of Osaka
2 時間で「チームに参加できる」状態へ

ゴール

経験者に助けてもらいながら、
AI も使いつつ、

**最低限チーム開発に参加できる
状態になる。**

今日は全部理解しなくていい



「何となく全体像が見える」
「困ったときに助けを求められる」
「PR を出す流れを知る」

これで合格。

今日のスコープ

やること

- Web アプリの**地図**を持つ
- **1回 PR** を出す
- エラーの**見方**を知る
- **AI / 経験者**への聞き方を知る

やらないこと

- React を深く学ぶ
- DB を設計する
- Docker を使いこなす
- きれいな設計を学ぶ

2026 年の優先順位

HTML / CSS 文法より先に、これらを身につけよう。

- AI への質問能力
- エラー共有能力
- 状況説明能力

コードを書ける < 助けを受けて安全に開発できる

全体構成（120 分）

#	内容	時間
1	Web アプリ全体像	15 分
2	開発フロー体験	45 分
3	Git 事故対策	15 分
4	DevTools & ターミナル	20 分
5	AI への聞き方	15 分
6	チームルール & まとめ	10 分

事前準備（できれば前日まで）

研修中に環境構築すると時間が崩れます。

- **GitHub** アカウント作成 / ログイン確認
- **Git** (`git --version` で確認)
- **Node.js** (`node --version` で確認)
- **VS Code** を起動できる
- **ターミナル**を開ける

01. Web アプリ全体像

15 分

このセクションのゴール

詳細実装ではなく、

自分がどこを触っているか分かる

状態を目指す。

Web アプリは「往復」で動く

- ユーザーの操作が API として送られる
- サーバーが処理し、DB に読み書き
- 結果が画面に戻る

データは流れっぱなしではなく、**行って帰ってくる。**

Frontend の役割

ユーザーが**直接**触れる部分。

- 画面の表示
- ボタンのクリック
- フォームの入力

例: React, HTML / CSS, JavaScript

Backend の役割

Frontend からの依頼を処理する。

- データ処理
- DB / Storage との接続
- ログイン / API 提供

例: Node.js, Python, Go

DB（データベース）

データを保存する場所。

- ユーザー情報
- 投稿内容
- 設定

消えると困るものはDB へ。

GitHub の役割

チーム開発の中心地。

- コードの共有
- PR レビュー
- 変更履歴の管理

Docker（概念だけ）

みんなの開発環境を揃える箱。

- 「うちでは動くのに…」を防ぐ
- 今は使い方を覚えなくて OK
- 「箱で揃える仕組みがある」だけ知る

02. 開発フロー体験

45 分 (中核)

このセクションのゴール

全員で実際に手を動かす。

- clone → branch → 編集 → commit → push → PR
- 初心者最大の壁「変更 → PR」を経験する
- 経験者は隣でサポート

今日の開発フロー

7ステップを一通り体験。

```
clone → branch → 編集 →  
commit → push → PR → review
```

詰まりやすいのは *clone* と *push*。すぐ呼んで！

Step 1. clone する

リポジトリを自分の PC に持ってくる。

```
git clone https://github.com/xxx/yyy.git  
cd yyy
```

URL はチームから配布されます。

Step 2. ブランチを切る

`main` で作業しない。最初に自分のブランチを作る。

```
git switch -c feature/first-pr
```

`git status` で今いるブランチが見える。

Step 3. 起動する

依存パッケージを入れて開発サーバーを立ち上げる。

```
npm install  
npm run dev
```

ブラウザで <http://localhost:3000> を開く。

⚠️ スクリーンショット

`npm run dev` 成功時のターミナル出力

(プレースホルダー)

Step 4. 文言を修正

例: ページのタイトルを変えてみよう。

```
- <h1>Hello</h1>  
+ <h1>Hello World</h1>
```

ブラウザをリロードして変わっているか確認！

Step 5. commit する

変更内容を Git に記録する。

```
git add .  
git commit -m "fix: change title text"
```

コミットメッセージは「**何をしたか**」を短く。

Step 6. push する

自分の変更を GitHub に送る。

```
git push -u origin feature/first-pr # 初回  
git push                            # 2 回目以降
```

初回は `-u origin <ブランチ名>` が必要。

⚠️ スクリーンショット

GitHub の Pull Request 作成画面

(プレースホルダー)

Step 7. PR を作成

GitHub 上で **Pull Request** を作る。

- ブラウザで GitHub を開く
- 「Compare & pull request」 ボタン
- タイトルと説明を書いて作成

03. Git 事故对策

15 分

このセクションのゴール

完璧に使いこなすより、まず

事故りやすいポイントを知る。

最低限の安全策を学ぶ。

ルール 1. main で作業しない

必ずブランチを切ってから作業。

```
git switch -c feature/my-task
```

feature/<やること> の形が分かりやすい。

ルール 2. 作業前に pull

最新を取ってきてから作業を始める。

```
git switch main  
git pull  
git switch -c feature/next-task
```

古い状態で作業すると *conflict* の原因。

ルール 3. 困ったら git status

今の状態を確認するクセを付ける。

```
git status
```

- どのファイルを変えたか
- どこまで commit したか
- どのブランチにいるか

ルール 4. conflict は助けを呼ぶ

merge conflict は初心者の鬼門。

- 無理に直そうとしない
- そのまま経験者に画面を見せる
- 学ぶのは慣れてから

禁止事項

これだけは絶対やらない。

```
git push --force          # ✗ 履歴を吹き飛ばす  
git push --force-with-lease # ✗ こちらもまず使わない
```

やるなら**必ず**経験者に確認。

04. DevTools & ターミナル

20 分

なぜ大事？

AI 活用の前提知識。

- エラーが読めれば AI に渡せる
- 渡せれば直せる
- 直せれば自走できる

DevTools の開き方

どのブラウザでも共通。

- **F12** キー
- 右クリック → 「検証」
- macOS: **Cmd + Option + I**

困ったとき → どのタブ？

症状	開くタブ
画面が真っ白	Console
ボタンを押しても保存されない	Network
見た目が崩れている	Elements

症状から逆引きできるようになるう。

Console タブ

JavaScript のエラーと出力を見る。

- 赤文字を探す
- まずは 1 行目を読む
- 例: `Uncaught TypeError: ...`

⚠️ スクリーンショット

Console タブにエラーが表示された画面
(プレースホルダー)

Network タブ

API 通信の様子を見る。

- リクエスト一覧
- **赤色 / 404 / 500** を探す
- 「サーバーまで届いてる？」を確認

⚠️ スクリーンショット

Network タブで失敗リクエストが赤く出ている画面
(プレースホルダー)

Elements タブ

HTML の構造を見る。

- 今表示されている HTML
- CSS が当たっているか
- クラス名は何か

最重要ポイント

エラー文を読む

文化を作ろう。

「赤いの出てます」ではなく、「〇〇というエラーが出ました」。

ターミナルは怖くない

初心者はターミナル恐怖症になりやすい。

まず覚えてほしいことは1つ:

壊れても大抵戻せる

最低限コマンド

コマンド	役割
<code>pwd</code>	今いる場所
<code>ls / dir</code>	ファイル一覧 (Win は dir)
<code>cd folder</code>	移動
<code>cd ..</code>	一つ上に戻る
<code>Ctrl + C</code>	動いているコマンドを止める

Ctrl + C を知るだけで安心感が違う。

05. AI への聞き方

15 分

なぜ必須？

2026 年において、AI を使いこなせるかは
生産性に直結するスキル。

- 最初の取っかかりは AI に聞いて OK
- ただし最後は**公式ドキュメント / 実行結果 / 経験者で確認**
- 聞き方で結果が大きく変わる

悪い例

動きません

- AI もエスパーではない
- 状況不明 → 当てずっぽうの回答
- 結局解決しない

良い例

React で npm run dev したら
Module not found が出ました

【やりたいこと】

ログイン画面を表示したい

【エラー全文】

XXX

【試したこと】

npm install をやり直した

AI 質問テンプレ（コピペ可）

【やりたいこと】

例： ログイン画面を表示したい

【起きていること】

例： `npm run dev` でエラー

【エラー全文】

（そのまま貼る）

【試したこと】

例： `npm install` をやり直した

【環境】

OS: macOS / Windows

ブラウザ: Chrome

スクショは要約しない

エラーを要約してしまうと伝わらない。

なんか赤いエラー出てます

→ AI には何も伝わらない。

スクショをそのまま貼る

AI は画像認識が強い。

- レイアウト崩れ
- エラー画面 / DevTools
- ターミナル

文字起こしより、画像をそのまま投げる方が早い。

スクショの撮り方

OS	範囲選択	全画面
Windows	Win + Shift + S	PrintScreen
macOS	Cmd + Shift + 4	Cmd + Shift + 3

実務では**範囲選択**が圧倒的に便利。

AI は間違えることがある

それっぽいコードを返すが、

- 存在しない関数を使う
- 古い API を提案する
- 副作用を説明しない

ことがある。

特に AI 任せにしない操作

- ライブラリの追加 / 削除
- DB の操作
- 設定ファイルの大きな変更
- `--force` 系コマンド

大きい変更の前は経験者に聞く。1 分の確認で半日が救える。

⚠ AI に貼ってはいけないもの

事故が起きやすい。投げる前に確認。

- パスワード / API キー
- `.env` の中身
- 個人情報（メール、住所、電話）
- 秘密の URL（社内 / 招待リンク）
- 未公開の本番データ

不安ならマスクしてから貼る。

06. チームルール & まとめ

10 分

なぜルールが必要？

学生プロジェクトはここで崩壊しやすい。

- 仲が悪いから崩れるのではない
- ルールがないから崩れる
- 最低限の約束を共有する

困ったときの判断フロー

1. エラー文を読む
2. スクショを撮る
3. AI に聞く（テンプレに沿って）
4. 10～30 分自分で試す
5. 進まなければ経験者に聞く

30 分で進まなければ即声をかける。

メンター相談テンプレ

【やりたいこと】
【起きていること】
【エラー全文】
【試したこと】
【スクショ】
【今いるブランチ】 ← Git 事故防止

ブランチ名を添えると、メンターの初動が速い。

チーム開発の最低ルール

- 30 分詰まったら相談
- 「動きません」ではなくエラー全文 + スクショ
- 勝手にライブラリ追加しない
- `main` に直 push しない
- 壊したら隠さない（次スライドで詳しく）

壊したら隠さない（最重要）

- 「やっちゃいました」を即報告
- 早ければ早いほどリカバリは簡単
- 隠すと被害が広がる

隠さないチームは強い。

今日「教えないこと」

触れない	理由
React 詳細 / TS 厳密 / CSS 設計	必要になったとき学ぶ方が定着
DB 理論 / Docker 操作	まず「参加できる」を優先
アルゴリズム / 設計原則	実戦で身につく

この研修の本質



重要なのは、
コードを書けることではなく、
助けを受けながら安全に開発参加できること。

質問タイム



ようこそ、チームへ！